```
#!/usr/bin/env python3
"""JAMB CBT Mobile Edition (Kivy Framework). Question bank and logic retained."""

import kivy
kivy.require('2.1.0')

from kivy.app import App
from kivy.uix.screenmanager import ScreenManager, Screen
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.gridlayout import GridLayout
from kivy.uix.label import Label
from kivy.uix.button import Button
from kivy.uix.scrollview import ScrollView
from kivy.clock import Clock

# Import your full question bank structure
from jamb_cbt_studio_gui import QUESTIONS_DATA, SUBJECT_ORDER

# Map subjects
SUBJECTS = {name: next(v['questions'] for v in QUESTIONS_DATA.values() if v['subject'] ==
name) for name in SUBJECT_ORDER}

class HomeScreen(Screen):
    def __init__(self, **kwargs):
        super(HomeScreen, self).__init__(**kwargs)
        layout = BoxLayout(orientation='vertical', padding=30, spacing=20)

        layout.add_widget(Label(text="JAMB CBT MOBILE", font_size='24sp', bold=True,
color=(0.09, 0.17, 0.30, 1)))
        layout.add_widget(Label(text="Select your mode to begin practice or exam simulation.",
font_size='14sp', color=(0.44, 0.50, 0.59, 1)))

        btn_exam = Button(text="Timed Examination", background_color=(0.04, 0.50, 0.42, 1),
font_size='16sp', bold=True)
        btn_exam.bind(on_press=lambda x: self.go_subjects('exam'))
        layout.add_widget(btn_exam)

        btn_practice = Button(text="Practice & Learn", background_color=(0.14, 0.27, 0.45, 1),
font_size='16sp', bold=True)
        btn_practice.bind(on_press=lambda x: self.go_subjects('practice'))
        layout.add_widget(btn_practice)

        self.add_widget(layout)
```

```
    def go_subjects(self, mode):
        app = App.get_running_app()
        app.mode = mode
        app.root.current = 'subjects'


class SubjectScreen(Screen):
    def __init__(self, **kwargs):
        super(SubjectScreen, self).__init__(**kwargs)
        self.layout = BoxLayout(orientation='vertical', padding=30, spacing=15)
        self.layout.add_widget(Label(text="CHOOSE A SUBJECT", font_size='20sp', bold=True,
color=(0.09, 0.17, 0.30, 1)))

        for name in SUBJECT_ORDER:
            b = Button(text=name, background_color=(0.04, 0.50, 0.42, 1), font_size='16sp',
bold=True)
            b.bind(on_press=lambda instance, n=name: self.start_exam(n))
            self.layout.add_widget(b)

        back_btn = Button(text="← Back to Home", background_color=(0.7, 0.7, 0.7, 1),
font_size='14sp')
        back_btn.bind(on_press=lambda x: setattr(App.get_running_app().root, 'current', 'home'))
        self.layout.add_widget(back_btn)

        self.add_widget(self.layout)

    def start_exam(self, subject):
        app = App.get_running_app()
        app.subject = subject
        app.questions = SUBJECTS[subject][:60]
        app.answers = {}
        app.flagged = set()
        app.idx = 0
        app.remaining = 3600

        exam_screen = app.root.get_screen('exam')
        exam_screen.load_question()
        app.root.current = 'exam'
        if app.mode == 'exam':
            exam_screen.start_timer()
```

```
class ExamScreen(Screen):
    def __init__(self, **kwargs):
        super(ExamScreen, self).__init__(**kwargs)
        self.timer_event = None

        root_layout = BoxLayout(orientation='vertical', padding=15, spacing=10)

        # Top bar: Progress & Timer
        top_bar = BoxLayout(size_hint_y=None, height=40)
        self.lbl_progress = Label(text="Q 1/60", bold=True, color=(0.04, 0.50, 0.42, 1))
        self.lbl_timer = Label(text="60:00", bold=True, color=(0.94, 0.31, 0.31, 1))
        top_bar.add_widget(self.lbl_progress)
        top_bar.add_widget(self.lbl_timer)
        root_layout.add_widget(top_bar)

        # Question text
        self.lbl_question = Label(text="", text_size=(350, None), halign='left', valign='top',
font_size='16sp', color=(0.09, 0.17, 0.30, 1))
        self.lbl_question.bind(size=lambda s, w: setattr(s, 'text_size', (w, None)))
        root_layout.add_widget(self.lbl_question)

        # Options container
        self.options_layout = BoxLayout(orientation='vertical', spacing=10)
        root_layout.add_widget(self.options_layout)

        # Navigation buttons (Touch friendly - Replaced arrow keys)
        nav_layout = BoxLayout(size_hint_y=None, height=50, spacing=10)
        btn_prev = Button(text="← Previous", background_color=(0.44, 0.50, 0.59, 1))
        btn_prev.bind(on_press=lambda x: self.previous_question())

        btn_next = Button(text="Next →", background_color=(0.04, 0.50, 0.42, 1))
        btn_next.bind(on_press=lambda x: self.next_question())

        nav_layout.add_widget(btn_prev)
        nav_layout.add_widget(btn_next)
        root_layout.add_widget(nav_layout)

        # Submit button
        btn_submit = Button(text="Submit Exam", size_hint_y=None, height=45,
background_color=(0.14, 0.27, 0.45, 1), bold=True)
        btn_submit.bind(on_press=lambda x: self.submit_exam())
        root_layout.add_widget(btn_submit)
```

```
        self.add_widget(root_layout)

    def load_question(self):
        app = App.get_running_app()
        q = app.questions[app.idx]
        self.lbl_progress.text = f"Q {app.idx + 1} / {len(app.questions)}"
        self.lbl_question.text = q['text']

        self.options_layout.clear_widgets()
        for key, val in sorted(q['options'].items()):
            selected = app.answers.get(app.idx) == key
            b = Button(text=f"{key}. {val}", background_color=(0.04, 0.50, 0.42, 1) if selected else
(0.9, 0.9, 0.9, 1), color=(1,1,1,1) if selected else (0,0,0,1))
            b.bind(on_press=lambda instance, k=key: self.select_answer(k))
            self.options_layout.add_widget(b)

    def select_answer(self, key):
        app = App.get_running_app()
        app.answers[app.idx] = key
        self.load_question()

    def previous_question(self):
        app = App.get_running_app()
        if app.idx > 0:
            app.idx -= 1
            self.load_question()

    def next_question(self):
        app = App.get_running_app()
        if app.idx < len(app.questions) - 1:
            app.idx += 1
            self.load_question()

    def start_timer(self):
        if self.timer_event:
            self.timer_event.cancel()
        self.timer_event = Clock.schedule_interval(self.update_timer, 1)

    def update_timer(self, dt):
        app = App.get_running_app()
        if app.remaining <= 0:
            self.submit_exam()
            return False
```

```
        app.remaining -= 1
        mins, secs = divmod(app.remaining, 60)
        self.lbl_timer.text = f"{mins:02d}:{secs:02d}"

    def submit_exam(self):
        if self.timer_event:
            self.timer_event.cancel()
        app = App.get_running_app()
        correct = sum(1 for i, q in enumerate(app.questions) if app.answers.get(i) == q['answer'])
        result_screen = app.root.get_screen('result')
        result_screen.set_result(correct, len(app.questions))
        app.root.current = 'result'


class ResultScreen(Screen):
    def __init__(self, **kwargs):
        super(ResultScreen, self).__init__(**kwargs)
        self.layout = BoxLayout(orientation='vertical', padding=30, spacing=20)
        self.lbl_score = Label(text="", font_size='22sp', bold=True, color=(0.09, 0.17, 0.30, 1))
        self.layout.add_widget(self.lbl_score)

        btn_home = Button(text="Back to Home", background_color=(0.04, 0.50, 0.42, 1),
font_size='16sp', bold=True)
        btn_home.bind(on_press=lambda x: setattr(App.get_running_app().root, 'current', 'home'))
        self.layout.add_widget(btn_home)

        self.add_widget(self.layout)

    def set_result(self, correct, total):
        pct = round((correct / total) * 100, 1) if total else 0
        self.lbl_score.text = f"Exam Complete!\n\nScore: {correct} / {total}\nPercentage: {pct}%"


class JambMobileApp(App):
    mode = "exam"
    subject = None
    questions = []
    answers = {}
    flagged = set()
    idx = 0
    remaining = 3600

    def build(self):
```

```
        sm = ScreenManager()
        sm.add_widget(HomeScreen(name='home'))
        sm.add_widget(SubjectScreen(name='subjects'))
        sm.add_widget(ExamScreen(name='exam'))
        sm.add_widget(ResultScreen(name='result'))
        return sm


if __name__ == '__main__':
    JambMobileApp().run()
```